Bioinformatics

2025/2026

Day 3

Exercise 1

Use conda to create an environment named bioinfo_lab3 containing Python 3.11.

Exercise 2

Manually align the following two sequences by creating a dot plot (with pen and paper or with a spreadsheet).

- **Sequence 1:** T A G C T A G C G A T C G A T (15 bases)
- **Sequence 2:** T A G C T A C C G A T C G A T (15 bases)

2.1 Create a dot plot by placing the Sequence 1 across the top and Sequence 2 down the left side. Place a dot (or an 'X') in every box where the row letter exactly matches the column letter. Draw a line through the longest continuous diagonal path you can find, starting from the top-left.

2.2 Interpretation: Does your main diagonal line go perfectly unbroken from the top-left to the bottom-right? Write down the exact coordinate (Row #, Column #) where the diagonal breaks. What biological event does this break represent? (Specify the exact letters involved).

2.3 Imagine you didn't want to draw all those scattered background dots, so you decided to use a sliding window of k=3 (3-mers) instead. How large would the "gap" in your new main diagonal be?

Exercise 3

Given the following two sequences:

- **Seq 1:** A T G C T A G
- **Seq 2:** A T C C T A G

3.1 Hamming Distance: Calculate the Hamming distance between Seq 1 and Seq 2.

3.2 Composition Vectors: Create the nucleotide counts [A, T, G, C] for both sequences.

3.3 Manhattan Distance: Calculate the Manhattan distance based on your composition vectors.

3.4 Euclidean Distance: Calculate the Euclidean distance based on your composition vectors.

Exercise 4

4.1 Consider the following sequence alignment and calculate the Percentage Identity, explicitly stating the values for Matches, Mismatches, Gaps, and the Total Alignment Length.

```
Seq A:  G  A  T  C  -  T  A
        |  |     |     |  |
Seq B:  G  A  A  C  G  T  A
```

4.2 Use the EMBOSS `needle` tool (Needleman-Wunsch algorithm) to globally align the above two sequences and eventually recreate the above alignment.

How: Write an executable shell script (e.g., `run_alignment.sh`) or a `Makefile` that fully automates this process without any manual user input. Your script must:

1. **Generate Data:** Programmatically create two FASTA files: `seqA.fasta` (`GATCTA`) and `seqB.fasta` (`GAACGTA`).
2. **Execute:** Run `needle` using the default parameters.
3. **Save:** Output the result to a file named `global_alignment.txt`.

Running your shell script or Makefile must successfully generate both the FASTA files and the `global_alignment.txt` results file from scratch.

Exercise 5

5.1 Write a Python function called `calculate_hamming_distance(seq1, seq2)` that takes two DNA strings as input and returns the Hamming distance as an integer.

- **Requirement:** Include an error check inside the function. If the sequences are not the exact same length, the function must raise a `ValueError` with a descriptive message.

5.2 Create a complete Python script named `hamming_distance.py`. This script should use your function from 5.1 to calculate the distance between two sequences entered by the user at the command line.

- **Requirements:**

- Accept inputs using command-line arguments: `--seq1 <sequence>` and `--seq2 <sequence>`. (Hint: Use the `argparse` module).
- Print *only* the final integer to standard output (`stdout`).
- If the sequences are empty strings or if they are of unequal length, the script should catch the error, print a helpful error message to standard error (`stderr`), and exit gracefully without crashing (with a non-zero exit status).

5.3 Testing: Use the hamming_distance.py script you created in 5.2 to calculate the Hamming distance for the three pairs of sequences listed below.

Your goal is to programmatically generate a results file named *hamming.csv* formatted with exactly two columns:

- Column 1: The name of the pair being tested (e.g., Pair A).
- Column 2: The resulting Hamming distance.
- **Crucial Rule:** If your Python script fails due to an error (such as the sequences being of unequal lengths in Pair C), your automated test must catch this and record a value of -1 in the CSV.

Do not create this CSV manually, write an executable shell script (e.g., run_tests.sh) or a Makefile that runs your Python script against all three pairs and generates the hamming.csv file automatically from scratch.

- **Pair A:**
  - Seq 1: `ATGCGTACGTAGCTA`
  - Seq 2: `ATGCCTACGTAGCTA`
- **Pair B:**
  - Seq 1: `GAGCCTACTAACGGGAT`
  - Seq 2: `CATCGTAATGACGGCCT`
- **Pair C (Should trigger an error):**
  - Seq 1: `ATGC`
  - Seq 2: `ATG`

Exercise 6

Write a Python function that takes a sequence and a window size *k* and returns a list of all k-mers.

For instance:

```
seq = "GATCGATC”
k = 3
print(get_kmers(seq, k))
# Should output: ['GAT', 'ATC', 'TCG', 'CGA', 'GAT', 'ATC']
```